Dependency Injection, SOLID Principles, Service Lifetime, OOPS, Static Class, Async & Await

> *A no-nonsense, story-driven guide to the concepts that every serious .NET developer must know cold.*
> *Read it once. Understand it forever.*

Table of Contents

---

1. Object-Oriented Programming (OOP)

The Big Picture

Think of OOP as building with Lego. Each Lego piece is a **class** — a blueprint. When you actually snap pieces together and build something, those are **objects** (instances of that class). The entire point of OOP is to model real-world things in code so the code becomes easier to read, reuse, and change.

C# is a fully object-oriented language. Everything lives inside a class.

The Four Pillars

1.1 Encapsulation — "Mind Your Own Business"

What it means: Bundle data and the methods that operate on that data together inside a class. Hide the internal details from the outside world.

The analogy: A TV remote. You press the volume button and the TV gets louder. You have no idea what happens inside the remote — and you do not need to. That is encapsulation.

```
public class BankAccount
{
    private decimal _balance; // private = hidden from outside

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            _balance += amount;
    }

    public decimal GetBalance()
    {
        return _balance;
    }
}
```

Nobody can directly touch `_balance` from outside the class. They have to go through `Deposit()` and `GetBalance()`. This prevents someone from doing `account._balance = -99999`, which would be a disaster.

Access Modifiers in C#:

Modifier	Who Can Access
`public`	Everyone
`private`	Only the same class
`protected`	Same class + subclasses
`internal`	Same assembly (project)
`protected internal`	Same assembly or derived class

1.2 Inheritance — "Children Inherit from Parents"

What it means: A class can inherit properties and methods from another class, avoiding code duplication.

The analogy: A `Dog` and a `Cat` are both `Animal`. Instead of writing "has a name", "can breathe", "can eat" for both, you write it once in `Animal` and both inherit it.

```
public class Animal
{
    public string Name { get; set; }

    public void Breathe()
    {
        Console.WriteLine($"{Name} is breathing.");
    }
}

public class Dog : Animal // Dog inherits from Animal
{
    public void Bark()
    {
        Console.WriteLine($"{Name} says: Woof!");
    }
}

public class Cat : Animal
{
    public void Meow()
    {
        Console.WriteLine($"{Name} says: Meow!");
    }
}
```

```
var dog = new Dog { Name = "Bruno" };
dog.Breathe(); // Inherited from Animal
dog.Bark();    // Dog's own method
```

Key Rule: C# only allows **single inheritance** for classes (a class can only have one direct parent). But a class can implement **multiple interfaces**.

1.3 Polymorphism — “One Interface, Many Forms”

What it means: The same method name behaves differently depending on the object it is called on.

The analogy: You say “speak” to a Dog and it barks. You say “speak” to a Cat and it meows. Same command, different behavior.

There are two types:

Compile-time Polymorphism (Method Overloading): Same method name, different parameters.

```
public class Calculator
{
    public int Add(int a, int b) => a + b;
    public double Add(double a, double b) => a + b;
    public int Add(int a, int b, int c) => a + b + c;
}
```

The compiler figures out which `Add` to call based on the arguments you pass.

Runtime Polymorphism (Method Overriding): A child class provides its own version of a method defined in the parent.

```
public class Animal
{
    public virtual void Speak()
    {
        Console.WriteLine("...");
    }
}

public class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Woof!");
    }
}

public class Cat : Animal
```

```
{
    public override void Speak()
    {
        Console.WriteLine("Meow!");
    }
}
```

```
Animal[] animals = { new Dog(), new Cat(), new Dog() };

foreach (var animal in animals)
{
    animal.Speak(); // Calls the correct version at runtime
}
// Output: Woof! Meow! Woof!
```

This is powerful. You are working with `Animal` references, but the right `Speak` is called based on the actual object type. This is the magic of runtime polymorphism.

1.4 Abstraction — "Show Only What Matters"

What it means: Hide complex implementation details. Expose only what the user of the class needs to know.

The analogy: A car's steering wheel. You know turning it left makes the car go left. You do not need to understand the hydraulic system, tie rods, and steering column to drive.

In C#, abstraction is achieved using **abstract classes** and **interfaces**.

Abstract Class: A class that cannot be instantiated on its own. It is a template that other classes must build upon.

```
public abstract class Shape
{
    public abstract double Area(); // No implementation here

    public void PrintArea()
    {
        Console.WriteLine($"Area: {Area()}"); // Uses child's implementation
    }
}
```

```
public class Circle : Shape
{
    private double _radius;
    public Circle(double radius) { _radius = radius; }

    public override double Area() => Math.PI * _radius * _radius;
}

public class Rectangle : Shape
{
    private double _width, _height;
    public Rectangle(double w, double h) { _width = w; _height = h; }

    public override double Area() => _width * _height;
}
```

Interface: A pure contract. No implementation, just method signatures. A class that implements an interface must provide the implementation.

```
public interface IPayable
{
    void ProcessPayment(decimal amount);
    bool ValidateCard(string cardNumber);
}

public class CreditCardPayment : IPayable
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing credit card payment: {amount}");
    }

    public bool ValidateCard(string cardNumber)
    {
        return cardNumber.Length == 16;
    }
}
```

Abstract Class vs Interface — When to use what:

Feature	Abstract Class	Interface
Can have implementation	Yes	No (before C# 8) / Default methods (C# 8+)
Can have fields	Yes	No
A class can inherit from	One only	Multiple
Use when	Sharing base behavior	Defining a contract

---

2. SOLID Principles

SOLID is an acronym for five design principles that make your code clean, maintainable, and extensible. Violate them and your codebase becomes a plate of spaghetti that nobody wants to touch.

S — Single Responsibility Principle (SRP)

"A class should have only one reason to change."

A class should do one thing and do it well. If a class is handling user login, sending emails, AND saving to the database, that is three reasons it could change. Split it up.

Bad:

```
public class UserService
{
    public void RegisterUser(string email, string password)
    {
        // Validate
        if (string.IsNullOrEmpty(email)) throw new Exception("Invalid email");

        // Save to DB
        var db = new SqlConnection("...");
        db.Execute("INSERT INTO Users ...");

        // Send welcome email
        var smtpClient = new SmtpClient();
```

```
        smtpClient.Send(email, "Welcome!", "Thanks for signing up.");
    }
}
```

This class knows too much. If the email service changes, you touch `UserService`. If the DB schema changes, you touch `UserService`. If validation rules change, you touch `UserService`.

Good:

```
public class UserValidator
{
    public bool IsValid(string email, string password) { /* ... */ }
}

public class UserRepository
{
    public void Save(User user) { /* save to DB */ }
}

public class EmailService
{
    public void SendWelcomeEmail(string email) { /* send email */ }
}

public class UserService
{
    private readonly UserValidator _validator;
    private readonly UserRepository _repo;
    private readonly EmailService _emailService;

    public UserService(UserValidator v, UserRepository r, EmailService e)
    {
        _validator = v; _repo = r; _emailService = e;
    }

    public void Register(string email, string password)
    {
        if (!_validator.IsValid(email, password)) throw new Exception("Invalid");
        _repo.Save(new User { Email = email });
```

```
        _emailService.SendWelcomeEmail(email);
    }
}
```

Now each class has one job. One reason to change.

O — Open/Closed Principle (OCP)

“Classes should be open for extension, but closed for modification.”

You should be able to add new functionality without touching existing, working code.

Bad: Every time a new payment type is added, you modify the existing class.

```
public class PaymentProcessor
{
    public void Process(string type, decimal amount)
    {
        if (type == "Credit") { /* credit logic */ }
        else if (type == "Debit") { /* debit logic */ }
        else if (type == "UPI") { /* Now you have to modify this class again */ }
    }
}
```

Good: Use abstraction. New payment types just add new classes.

```
public interface IPaymentStrategy
{
    void Pay(decimal amount);
}

public class CreditCardPayment : IPaymentStrategy
{
    public void Pay(decimal amount) => Console.WriteLine($"Credit card: {amount}");
}

public class UpiPayment : IPaymentStrategy
{
    public void Pay(decimal amount) => Console.WriteLine($"UPI: {amount}");
}
```

```
public class PaymentProcessor
{
    private readonly IPaymentStrategy _strategy;
    public PaymentProcessor(IPaymentStrategy strategy) { _strategy = strategy; }

    public void Process(decimal amount) => _strategy.Pay(amount);
}
```

Adding Bitcoin payment later? Just create a new class. Zero modifications to existing code.

L — Liskov Substitution Principle (LSP)

"If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering the correctness of the program."

In plain English: If you have a `Bird` class and a `Penguin` subclass, you should be able to use a `Penguin` wherever a `Bird` is expected — and nothing should break.

Bad violation:

```
public class Bird
{
    public virtual void Fly() => Console.WriteLine("Flying...");
}

public class Penguin : Bird
{
    public override void Fly()
    {
        throw new NotImplementedException("Penguins cannot fly!"); // VIOLATION
    }
}
```

This breaks the contract. Anyone using `Bird` objects now has to worry about whether their bird can actually fly.

Good:

```
public abstract class Bird
{
    public abstract void Move();
}

public class FlyingBird : Bird
{
    public override void Move() => Console.WriteLine("Flying...");
}

public class Penguin : Bird
{
    public override void Move() => Console.WriteLine("Swimming...");
}
```

The hierarchy now reflects reality. A `Penguin` is a `Bird`, it just moves differently.

I — Interface Segregation Principle (ISP)

“Clients should not be forced to depend on interfaces they do not use.”

Do not create fat interfaces. Split them into smaller, focused ones.

Bad:

```
public interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
}

public class Robot : IWorker
{
    public void Work() => Console.WriteLine("Working...");
    public void Eat() => throw new NotImplementedException(); // Robots don't eat
    public void Sleep() => throw new NotImplementedException(); // Robots don't sleep
}
```

The `Robot` is being forced to implement methods it does not need.

Good:

```
public interface IWorkable { void Work(); }
public interface IEatable { void Eat(); }
public interface ISleepable { void Sleep(); }

public class Human : IWorkable, IEatable, ISleepable
{
    public void Work() => Console.WriteLine("Working...");
    public void Eat() => Console.WriteLine("Eating...");
    public void Sleep() => Console.WriteLine("Sleeping...");
}

public class Robot : IWorkable
{
    public void Work() => Console.WriteLine("Working 24/7...");
}
```

D — Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

Your business logic should not be tightly coupled to specific implementations like a particular database, logger, or email provider.

Bad: `OrderService` directly depends on `SqlOrderRepository`. Swap to MongoDB? Rewrite `OrderService`.

```
public class OrderService
{
    private SqlOrderRepository _repo = new SqlOrderRepository(); // Tight coupling

    public void PlaceOrder(Order order)
    {
        _repo.Save(order);
    }
}
```

Good: Both depend on the abstraction `IOrderRepository`.

```
public interface IOrderRepository
{
    void Save(Order order);
}

public class SqlOrderRepository : IOrderRepository
{
    public void Save(Order order) { /* SQL logic */ }
}

public class MongoOrderRepository : IOrderRepository
{
    public void Save(Order order) { /* MongoDB logic */ }
}

public class OrderService
{
    private readonly IOrderRepository _repo;

    public OrderService(IOrderRepository repo) // Injected from outside
    {
        _repo = repo;
    }

    public void PlaceOrder(Order order) => _repo.Save(order);
}
```

Now `OrderService` does not care whether data goes to SQL or MongoDB. This is exactly where **Dependency Injection** comes in (covered in detail later).

---

3. Static Classes and the `static` Keyword

What Does `static` Mean?

When something is `static`, it belongs to the **type itself**, not to any particular instance. It exists once, shared across the entire application, from the moment the class is first used until the application exits.

Think of it like a building's address board. Every resident (instance) comes and goes, but the board itself is fixed to the building (the class).

Static Methods

```
public class MathHelper
{
    public static int Square(int n) => n * n;
    public static double CircleArea(double r) => Math.PI * r * r;
}

// No object needed
int result = MathHelper.Square(5); // 25
```

You call it on the class name directly. No `new MathHelper()` required.

Static Fields and Properties

```
public class Counter
{
    private static int _count = 0; // Shared across ALL instances

    public Counter()
    {
        _count++;
    }

    public static int GetCount() => _count;
}

var c1 = new Counter();
var c2 = new Counter();
var c3 = new Counter();

Console.WriteLine(Counter.GetCount()); // 3 — all three instances share _count
```

Static Classes

A **static class** cannot be instantiated at all. Every member must be static. It is a container for utility functions and constants.

```
public static class StringUtils
{
    public static string Capitalize(string s)
    {
        if (string.IsNullOrEmpty(s)) return s;
        return char.ToUpper(s[0]) + s.Substring(1);
    }

    public static bool IsPalindrome(string s)
    {
        var reversed = new string(s.Reverse().ToArray());
        return s.Equals(reversed, StringComparison.OrdinalIgnoreCase);
    }
}
```

```
// new StringUtils() — this will NOT compile. Static classes cannot be instantiated.
Console.WriteLine(StringUtils.Capitalize("hello")); // Hello
Console.WriteLine(StringUtils.IsPalindrome("racecar")); // True
```

Rules for static classes:

- Cannot be instantiated (`new` will not compile)
- Cannot be inherited
- All members must be `static`
- They are `sealed` implicitly

Static Constructors

A static constructor runs **once**, automatically, before the first use of the class. You cannot call it manually, and it takes no parameters.

```
public class AppConfig
{
    public static string ConnectionString { get; }
```

```
    static AppConfig() // Runs once, automatically
    {
        ConnectionString = Environment.GetEnvironmentVariable("DB_CONN")
                           ?? "default_connection_string";
        Console.WriteLine("AppConfig initialized.");
    }
}
```

Static in Context: Extension Methods

Extension methods are a brilliant use of `static`. They let you "add" methods to existing types without modifying them.

```
public static class StringExtensions
{
    public static bool IsNullOrEmpty(this string s)
    {
        return string.IsNullOrEmpty(s);
    }

    public static string ToTitleCase(this string s)
    {
        return System.Globalization.CultureInfo.CurrentCulture
              .TextInfo.ToTitleCase(s.ToLower());
    }
}
```

```
string name = "john doe";
Console.WriteLine(name.ToTitleCase()); // "John Doe"
// Called as if it is a method ON the string itself
```

When to Use Static vs Instance — The Decision Rule

Use `static` when	Use instance when
The method has no state to maintain	The behavior depends on object state

It is a pure utility function (like Math.Sqrt)	The class represents an entity with data
It is an extension method	You need polymorphism or inheritance
It is a constant or config shared globally	The lifecycle of data needs to be controlled

Warning: Overusing `static` leads to hidden global state, which makes testing and debugging a nightmare. Use it deliberately.

---

4. Async and Await

The Problem Without Async

Imagine you order food at a restaurant. A bad restaurant makes every other customer wait while your food is being cooked. A good restaurant takes your order, passes it to the kitchen, and serves other customers in the meantime. That good restaurant is running **asynchronously**.

In software terms: when your application calls a database query or an HTTP API, that operation takes time. Without async, your thread sits there doing nothing, blocking everything else. With async, the thread is freed to do other work while waiting.

The Basics

```
// Synchronous — blocks the thread while waiting
public string GetData()
{
    Thread.Sleep(3000); // Nothing else can happen for 3 seconds
    return "data";
}

// Asynchronous — thread is free while waiting
public async Task<string> GetDataAsync()
{
    await Task.Delay(3000); // Thread is released back to the pool
    return "data";
}
```

async marks a method as asynchronous. It tells the compiler to transform the method into a state machine.

await is where the magic happens. It says: “wait for this operation to complete, but do NOT block the thread. Free it up. Come back here when done.”

Task represents an ongoing operation. `Task<T>` is an operation that will eventually return a value of type `T`.

The Return Types

```
public async Task DoSomethingAsync()
{
    // No return value, but still async
    await Task.Delay(1000);
}

public async Task<int> GetNumberAsync()
{
    await Task.Delay(500);
    return 42; // Task<int> is automatically handled
}

public async ValueTask<string> GetCachedDataAsync(bool useCache)
{
    if (useCache) return "cached"; // No await needed, returns synchronously
    await Task.Delay(200);
    return "fresh data";
    // ValueTask is more efficient when the result is often available synchronously
}
```

A Real-World Example

```
public class WeatherService
{
    private readonly HttpClient _httpClient;

    public WeatherService(HttpClient client)
```

```
    {
        _httpClient = client;
    }

    public async Task<string> GetWeatherAsync(string city)
    {
        // This HTTP call could take 200ms–2000ms
        // Without async: thread blocked the entire time
        // With async: thread is free to serve other requests
        var response = await _httpClient.GetStringAsync($"https://api.weather.com/{city}");
        return response;
    }
}
```

Awaiting Multiple Tasks

Sequential (slow): One after the other.

```
public async Task SequentialExample()
{
    var user = await GetUserAsync();     // waits 500ms
    var orders = await GetOrdersAsync(); // then waits 300ms
    // Total: 800ms
}
```

Parallel (fast): Both kicked off at the same time.

```
public async Task ParallelExample()
{
    var userTask = GetUserAsync();     // Started
    var ordersTask = GetOrdersAsync(); // Started immediately, NOT awaited yet

    var user = await userTask;
    var orders = await ordersTask;
    // Total: ~500ms (whichever is longer)
}
```

Or with `Task.WhenAll`:

```
public async Task WhenAllExample()
{
    var (user, orders) = await Task.WhenAll(GetUserAsync(), GetOrdersAsync());
    // Cleaner syntax for parallel execution
}
```

ConfigureAwait

```
// In library code (non-UI), use ConfigureAwait(false)
// This tells the runtime: don't bother capturing the current context
// It is a performance optimization in library/service code
var data = await SomeOperationAsync().ConfigureAwait(false);
```

In ASP.NET Core you generally do not need `ConfigureAwait(false)` because there is no SynchronizationContext, but in libraries and Blazor components, it matters.

Common Mistakes

Mistake 1: Async void (except for event handlers)

```
// BAD — exceptions are swallowed, cannot be awaited
public async void DoSomethingAsync()
{
    await Task.Delay(1000);
    throw new Exception(); // This exception goes unhandled and crashes the app
}

// GOOD
public async Task DoSomethingAsync()
{
    await Task.Delay(1000);
    throw new Exception(); // This can be caught by the caller
}
```

Mistake 2: Blocking with .Result or .Wait()

```
// BAD — this is a deadlock waiting to happen in certain contexts
var result = GetDataAsync().Result;
var result2 = GetDataAsync().GetAwaiter().GetResult(); // Slightly better but still risky

// GOOD — await all the way up
var result = await GetDataAsync();
```

Mistake 3: Not awaiting at all

```
// This looks fine but the task is running "fire and forget" — you lose error handling
public async Task ProcessAsync()
{
    SaveToDatabase(); // If this throws, you will never know
}

// GOOD
public async Task ProcessAsync()
{
    await SaveToDatabase();
}
```

---

5. Dependency Injection (DI)

The Problem DI Solves

Without DI, classes create their own dependencies. This sounds normal until you realize it means:

- You cannot test a class in isolation (because it always drags its dependencies with it)
- Swapping implementations requires modifying source code
- Your code is tightly coupled and fragile

DI says: do not let the class create what it needs. Give it from the outside.

The Core Concept

```
// WITHOUT DI — tightly coupled
public class OrderService
{
    private EmailService _emailService = new EmailService(); // Created internally
    private SqlOrderRepository _repo = new SqlOrderRepository(); // Created internally

    public void PlaceOrder(Order order)
    {
        _repo.Save(order);
        _emailService.SendConfirmation(order.CustomerEmail);
    }
}
```

To test `OrderService`, you must have a working `EmailService` and database connection. Every test hits the real database and sends real emails. That is a disaster.

```
// WITH DI — loosely coupled
public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly IEmailService _emailService;

    // Dependencies are "injected" via constructor
    public OrderService(IOrderRepository repo, IEmailService emailService)
    {
        _repo = repo;
        _emailService = emailService;
    }

    public async Task PlaceOrderAsync(Order order)
    {
        await _repo.SaveAsync(order);
        await _emailService.SendConfirmationAsync(order.CustomerEmail);
    }
}
```

Now for testing you inject fakes. For production you inject real ones. Same class, different behavior.

How DI Works in ASP.NET Core

ASP.NET Core has a built-in **IoC (Inversion of Control) Container**. You register services in `Program.cs`, and the framework injects them automatically wherever they are needed.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// Registration
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
builder.Services.AddScoped<IEmailService, SmtpEmailService>();
builder.Services.AddScoped<OrderService>();

var app = builder.Build();
```

```
// The controller — ASP.NET Core injects OrderService automatically
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrdersController(OrderService orderService)
    {
        _orderService = orderService;
    }

    [HttpPost]
    public async Task<IActionResult> PlaceOrder([FromBody] Order order)
    {
        await _orderService.PlaceOrderAsync(order);
        return Ok();
    }
}
```

The framework sees that `OrdersController` needs `OrderService`. It creates one, sees that `OrderService` needs `IOrderRepository` and `IEmailService`, creates those, passes them in — all automatically. This is called **automatic dependency resolution**.

Constructor Injection vs Property Injection vs Method Injection

Constructor Injection (Preferred):

```
public class ProductService
{
    private readonly IProductRepository _repo;

    public ProductService(IProductRepository repo)
    {
        _repo = repo;
    }
}
```

Dependencies are required. The class cannot exist without them. This is the safest and most explicit pattern.

Property Injection (Optional dependencies):

```
public class ReportService
{
    public ILogger Logger { get; set; } // Optional: works with or without a logger
}
```

Use this when a dependency is truly optional and the class can function without it.

Method Injection (One-time dependency):

```
public class DataProcessor
{
    public void Process(IDataSource source, DataRequest request)
    {
        // source is only needed for this one call
    }
}
```

Use this when a dependency is only needed for one specific operation.

---

6. Service Lifetime in DI

When you register a service with the DI container, you also define **how long it lives**. This is one of the most misunderstood and most important concepts in ASP.NET Core.

There are three lifetimes: **Singleton**, **Scoped**, and **Transient**.

Transient — "Born fresh every time"

A new instance is created **every single time** the service is requested from the container.

```
builder.Services.AddTransient<IEmailSender, EmailSender>();
```

Timeline: Created on every `GetService<>()` call. Disposed immediately after use.

Use when: The service is lightweight, stateless, and cheap to create. You want a fresh slate every time.

```
public class GuidGenerator
{
    public string Generate() => Guid.NewGuid().ToString();
}

// Registered as Transient
// Every time GuidGenerator is injected, it is a brand new object
// So each caller gets their own instance
```

Scoped — "One per request"

A new instance is created **once per HTTP request** (or per scope). Everyone within the same request gets the same instance.

```
builder.Services.AddScoped<IOrderRepository, SqlOrderRepository>();
```

Timeline: Created when an HTTP request begins. Disposed when the request ends.

Use when: The service holds state that should be consistent for one request (like a database context). `DbContext` in Entity Framework Core is the classic example.

```
// In a single HTTP request:
// Controller -> OrderService -> OrderRepository
```

```
// All three get the SAME DbContext instance
// This is critical for transactions — all operations share the same connection
builder.Services.AddScoped<AppDbContext>();
```

Singleton — “One for life”

A single instance is created for the **entire lifetime of the application**. Every request, every class, every thread shares the exact same instance.

```
builder.Services.AddSingleton<IMemoryCache, MemoryCache>();
builder.Services.AddSingleton<AppConfiguration>();
```

Timeline: Created once on first request. Lives until the application shuts down.

Use when: The service is expensive to create, stateless (or its state is thread-safe), and meant to be shared globally. Examples: configuration, caching, HTTP clients, connection pools.

The Critical Rule: Captive Dependency

You must never inject a shorter-lived service into a longer-lived one.

Injecting into	Can receive	Cannot receive
Singleton	Singleton only	Scoped, Transient
Scoped	Singleton, Scoped	Transient (careful)
Transient	Singleton, Scoped, Transient	Nothing restricted

Why is this a problem?

```
// WRONG
public class MySingleton
{
    private readonly IScopedService _scoped;

    public MySingleton(IScopedService scoped) // This is injected at startup
    {
        _scoped = scoped;
```

```
        // This scoped service instance is now CAPTURED forever in the singleton
        // It will never be disposed, and it will be shared across ALL requests
        // This is called "Captive Dependency" — a serious bug
    }
}
```

The scoped service is supposed to be unique per request. But since the singleton holds onto it, the same instance is shared across all requests. If the scoped service holds a database connection, you now have concurrent requests fighting over the same connection. Data corruption, threading bugs, unpredictable behavior.

ASP.NET Core will detect this and throw an `InvalidOperationException` in Development mode.

Visual Summary of Lifetimes

```
Application Start ----------------------------------------> Application End
|
|  Singleton: [=====================================================]  (1 in
|
|  Request 1: [Scoped:====]
|  Request 2:            [Scoped:====]
|  Request 3:                        [Scoped:====]           (1 per request)
|
|  Transient: [T][T]      [T][T][T]     [T]                  (new every time)
```

Registering Multiple Implementations

```
// Register multiple implementations of the same interface
builder.Services.AddScoped<INotificationService, EmailNotificationService>();
builder.Services.AddScoped<INotificationService, SmsNotificationService>();

// Inject all of them
public class NotificationDispatcher
{
    private readonly IEnumerable<INotificationService> _services;

    public NotificationDispatcher(IEnumerable<INotificationService> services)
```

```
    {
        _services = services;
    }

    public async Task NotifyAllAsync(string message)
    {
        foreach (var service in _services)
        {
            await service.SendAsync(message);
        }
    }
}
```

---

7. Interview Questions and Answers

OOP

Q: What is the difference between an abstract class and an interface in C#?

An abstract class can have implemented methods, fields, constructors, and access modifiers. A class can inherit from only one abstract class. An interface (pre C# 8) is a pure contract with no implementation, and a class can implement multiple interfaces. Use an abstract class when you want to share base behavior. Use an interface when you want to define a contract that multiple unrelated classes can fulfill.

Q: Can you instantiate an abstract class?

No. An abstract class cannot be instantiated directly. You must create a concrete subclass that implements all abstract members, and instantiate that instead.

Q: What is the difference between method overloading and method overriding?

Overloading is compile-time polymorphism — same method name in the same class but with different parameter signatures. The compiler decides which one to call. Overriding is runtime polymorphism — a derived class provides its own implementation of a `virtual` or `abstract` method from the base class. The runtime decides which one to call based on the actual object type.

Q: What is the `sealed` keyword?

`sealed` on a class prevents it from being inherited. `sealed` on a method in a derived class prevents further overriding. It is used when you want to stop the inheritance chain.

```
public sealed class FinalClass { }  // Cannot be inherited
// public class Child : FinalClass { } — compile error
```

SOLID

Q: Explain the Single Responsibility Principle with an example.

SRP states a class should have only one reason to change. If an `InvoiceService` generates invoices, prints them, and saves them to a database, it has three responsibilities and three reasons to change. Split it into `InvoiceGenerator`, `InvoicePrinter`, and `InvoiceRepository`. Now each changes only when its specific responsibility changes.

Q: What is the Dependency Inversion Principle and how does it relate to Dependency Injection?

DIP says high-level modules (business logic) should not depend on low-level modules (database, email). Both should depend on abstractions (interfaces). Dependency Injection is the practical mechanism to achieve DIP — instead of a class creating its own dependencies, they are injected from outside, typically through the constructor. This decouples implementation details from business logic.

Q: How does the Open/Closed Principle help in a real project?

When requirements change (which they always do), OCP lets you add new behavior by adding new code rather than modifying existing, tested code. For example, if you have a report generator and a new report format is needed, you add a new class implementing `IReportFormatter` rather than adding an `if/else` to the existing class. This reduces the risk of introducing bugs in working code.

Static

Q: What is the difference between a static class and a singleton?

A static class cannot be instantiated and all its members are static. It is best for stateless utility functions. A singleton is a design pattern where a class has one instance managed through DI or a static property, but the class itself is a normal class. Singletons can

implement interfaces, participate in polymorphism, and be swapped for testing. Static classes cannot. In modern ASP.NET Core, prefer `AddSingleton<T>()` over static classes for shared services.

Q: Can a static method access instance members?

No. A static method belongs to the class, not to any instance. It has no access to `this`, and therefore cannot access instance fields or methods directly. To use instance members from a static method, you would need to pass an instance as a parameter.

Q: When would you choose a static method over an instance method?

When the method does not depend on any instance state and is a pure function or utility (like `Math.Sqrt` or `string.IsNullOrEmpty`). If the result depends purely on the input parameters and nothing else about the object, it is a good candidate for `static`.

Async / Await

Q: What is the difference between `Task.Wait()` and `await`?

`Task.Wait()` is a synchronous blocking call — it blocks the calling thread until the task completes. `await` is asynchronous — it suspends the current method and releases the thread back to the pool until the task completes, then resumes. Using `.Wait()` can cause deadlocks in certain contexts (like old ASP.NET or UI threads with a SynchronizationContext).

Q: What happens when you use `async void`?

`async void` should be avoided except for event handlers. The problem is that exceptions thrown inside an `async void` method cannot be caught by the caller and will crash the application. Additionally, the caller has no way to `await` the operation, so you lose the ability to track when it completes.

Q: What is `ConfigureAwait(false)` and when should you use it?

By default, after an `await`, execution resumes on the original synchronization context (e.g., the UI thread or the ASP.NET request context). `ConfigureAwait(false)` tells the runtime not to capture the context — continue on whatever thread is available. This avoids context-switching overhead. Use it in library code that does not need to return to a specific context. In ASP.NET Core application code, it is generally not needed because there is no SynchronizationContext.

Q: What is `Task.WhenAll` and when do you use it?

`Task.WhenAll` takes multiple tasks and returns a new task that completes when ALL the input tasks have completed. Use it when you have multiple independent async operations that can run concurrently. For example, fetching user data and order data simultaneously. It is significantly faster than awaiting them one by one.

Dependency Injection

Q: What are the three ways to inject dependencies in ASP.NET Core?

Constructor injection (most common and recommended — dependencies are required), property injection (optional dependencies), and method injection (dependency needed only for a specific method call). ASP.NET Core's DI container natively supports constructor injection.

Q: What is the difference between `AddScoped`, `AddTransient`, and `AddSingleton`?

`AddTransient` creates a new instance every time the service is requested. `AddScoped` creates one instance per HTTP request — the same instance is shared within a single request but different requests get different instances. `AddSingleton` creates one instance for the entire application lifetime, shared across all requests and all services.

Q: What is a captive dependency and why is it a problem?

A captive dependency occurs when a service with a longer lifetime holds a reference to a service with a shorter lifetime — most commonly, a Singleton holding a Scoped service. The Scoped service gets "captured" and lives as long as the Singleton, meaning it is never properly disposed and is shared across all requests. This can cause data leaks, threading bugs, and inconsistent state. ASP.NET Core throws an `InvalidOperationException` at startup when it detects this in the Development environment.

Q: How do you register multiple implementations of the same interface?

Register them multiple times with `AddScoped` (or other lifetime). To consume all implementations, inject `IEnumerable<IYourInterface>` in the constructor. To consume only the last registered one, inject `IYourInterface` directly (ASP.NET Core resolves to the last registered implementation by default).

Q: What is the difference between the DI container and a Service Locator?

Both resolve dependencies, but in opposite directions. With DI (specifically constructor injection), dependencies are pushed into the class from outside — the class declares what it needs, and the container provides it. With a Service Locator, the class actively pulls its dependencies by calling the container inside its own code (`serviceLocator.Get<IMyService>()`). Service Locator is considered an anti-pattern because it hides dependencies, makes testing harder, and couples your code to the container itself.

Quick Reference Cheat Sheet

SOLID in One Line Each

Principle	One-liner
Single Responsibility	One class, one job
Open/Closed	Add new code, do not modify old code
Liskov Substitution	A subclass must be usable wherever its parent is used
Interface Segregation	Prefer many small interfaces over one large one
Dependency Inversion	Depend on abstractions, not concrete implementations

Service Lifetimes at a Glance

Lifetime	Created	Disposed	Best for
Transient	Every request	After use	Lightweight, stateless services
Scoped	Once per HTTP request	End of request	DbContext, request-specific data
Singleton	Once at startup	App shutdown	Config, caching, HTTP clients

Async Quick Rules

Rule	Why

Never use `async void` (except events)	Exceptions are unhandled, cannot be awaited
Never use `.Result` or `.Wait()`	Deadlock risk
Use `await Task.WhenAll()` for parallel ops	Faster than sequential awaiting
Use `ConfigureAwait(false)` in libraries	Avoids unnecessary context switching
Return `Task<T>` not `T` from async methods	Allows proper awaiting up the call chain

---

This document covers the core of what it means to write clean, production-grade C# code. Master these and you will write code that other developers actually want to maintain.